SuperChat

A single conversational assistant combining Aviasales (flights), the Shopping Assistant (e-commerce) and CinemaPlus (cinema)

Phase 1	Individual competition build on mock data
Phase 2	Integration with real APIs after the competition

1. Background

Three products currently exist as separate chat experiences. **Aviasales** helps users search and choose flights. The **Shopping Assistant** is an e-commerce assistant (for example, one serving a marketplace such as Birmarket) that helps users find, compare and choose products. **CinemaPlus** helps users discover films, check showtimes and book tickets. A user who needs more than one of these today must switch between applications and repeat context each time.

The goal of this project is to build **SuperChat**: one chat window in which the user can talk about anything covered by any of the three products, in any order, without choosing a mode. The system must decide, message by message, which domain should handle the request and must keep the conversation coherent when the topic changes.

2. Objectives

- Combine the three assistants behind one conversational interface.
- Route each user message to the correct domain automatically, including when the topic switches mid-conversation.
- Preserve context within each domain and across the whole session.
- Design the system so that mock data sources can be replaced by real APIs with minimal code change.
- Handle ambiguity, mixed requests and out-of-scope input gracefully.

3. Core Functional Requirements

3.1 Single chat interface

The product must expose one chat (web, mobile web or desktop; the participant decides). The interface shows the message history of the entire session regardless of which domain answered. Users never select a domain manually. The interface may, but is not required to, indicate which domain handled a reply.

3.2 Routing

Every incoming message is classified into one of the following categories before a response is produced.

Category	Description	Expected behaviour
Flights	Searching or comparing flights, dates, prices, airlines, baggage, flight booking	Handled by the Flights module

Category	Description	Expected behaviour
Shopping	Searching, filtering, comparing or asking about products, prices, stock, cart	Handled by the Shopping module
Cinema	Films, genres, cinemas, showtimes, seat availability, ticket booking	Handled by the Cinema module
Mixed	One message contains two or more intents (for example, a flight and a backpack for the trip)	All relevant modules are called; one coherent reply is produced
Follow-up	Short reply that depends on earlier context ("the cheaper one", "and for Saturday?")	Resolved against the active domain and recent turns
Ambiguous	Cannot be assigned confidently (for example, "show me something good" or "book it")	System asks one short clarifying question
Out of scope	Unrelated to flights, shopping or cinema	Polite refusal with a pointer to what the assistant can do

Participants are free to choose the routing technique (LLM function calling, a dedicated classifier, embeddings, rules, or a hybrid). The chosen approach and its trade-offs must be explained in the technical report.

3.3 Conversation state

- Maintain separate state per domain (for example, current flight search and selected flight, product shortlist and cart, selected film and showtime).
- Maintain shared state for the session (language, preferences stated so far, last active domain).
- A user must be able to return to a previous topic ("back to the headphones") and find the earlier state intact.

3.4 Flights module (minimum)

- Search flights by origin, destination and date; support one-way requests and, optionally, round trips.
- Filter and sort by price, number of stops, departure time and airline.
- Show flight details: times, duration, stops, baggage allowance and seats left.
- Compare two or more flights.
- Reserve a flight (mock reservation, no payment).

3.5 Shopping module (minimum)

- Search products by text, category, brand, price range, colour.
- Show product details and compare two or more products.
- Report availability from stock data.
- Add to and remove from a cart; show the cart total.

3.6 Cinema module (minimum)

- List films by genre, rating, language or what is showing on a given date.
- Show cinemas and showtimes for a film.
- Report seat availability and ticket price.

- Reserve tickets for a showtime (mock reservation, no payment).

3.7 Quality expectations

- The assistant must not invent flights, products, films, prices or showtimes that are absent from the data. If nothing matches, it says so.
- Responses are concise and written in the language of the user.
- Errors from data sources are handled without exposing stack traces or breaking the chat.

4. Phases

Phase 1: Competition (mock data)

Each participant builds individually against the mock dataset supplied by the organisers (see Appendix A for structure). Data access must be wrapped behind an interface layer (for example, `FlightRepository`, `ProductRepository` and `ShowtimeRepository`) so that the source can be swapped. Participants may extend the dataset for their own testing; the official dataset is the reference.

Phase 2: Production data (after the competition)

The mock layer is replaced with the real Aviasales, Shopping Assistant (e-commerce backend) and CinemaPlus APIs. Documentation and credentials will be provided by the organisers at that stage. Work in this phase includes authentication, rate limits, pagination, latency handling, caching, and logging. A clean data interface built in Phase 1 makes this phase substantially easier.

5. Bonus Tasks

Bonus tasks are optional and are considered only when the core requirements work reliably.

Bonus	Description
Voice input (Shopping Assistant)	The user can speak a request. Speech is transcribed and enters the same routing pipeline. Optional spoken replies. Voice activity detection (start and end of speech) should work without a push-to-talk button.
Visual search (Shopping Assistant)	The user uploads a photo of a product; the system identifies the item or category and returns matching products from the catalogue. Must state its confidence and fall back to a question when unsure.
Multilingual support (whole SuperChat)	Full conversation in Azerbaijani, Russian and English across all three domains, including switching language mid-session and mixed-language queries. Routing accuracy should be comparable across languages.

6. Deliverables

Item	Content
Source code	Repository with README, setup instructions and a single command to run the application
Technical report	Maximum 4 pages: architecture, routing method, state handling, data layer design, known limitations

Item	Content
Demo	Live demonstration of 5 minutes, including at least one conversation that switches domains three or more times
Test log	Results of your own routing tests with accuracy figures and a short error analysis

7. Sample Test Conversations

These conversations show the kind of behaviour expected. Participants should build their own larger set. The reference date of the mock dataset is Monday, 2 November 2026; expressions such as "tonight", "Friday" or "Saturday" are resolved against it.

#	Conversation	What it tests
1	User: "Find me a flight from Baku to Istanbul on Friday." then "What is playing tonight?" then "Do you have wireless headphones under 100 AZN?"	Switch across all three domains
2	User: "What is playing tonight?" then "Show me laptops." then "Okay, and the sci-fi one, any seats on Saturday?"	Domain switch and returning to Cinema; resolving "the sci-fi one"
3	User: "Show me laptops." then "The second one, is it in stock?"	Follow-up within Shopping
4	User: "Flights to Istanbul on 6 November." then "The cheaper one, is it direct?"	Follow-up within Flights
5	User: "I need a flight to Istanbul on 6 November and a backpack for the trip."	Mixed intent in one message
6	User: "Show me something good."	Ambiguity: must ask a clarifying question
7	User: "What is the weather in Ganja?"	Out of scope
8	User: "Book two seats." (no earlier context)	Missing context: must ask which film and showtime, or which flight

8. Rules and Constraints

- The project is individual work. Each participant must be able to explain any part of their submission during the demo.
- Any language model, framework or open-source library may be used, provided the licence allows it and sources are cited in the report.
- No real personal data or real payment details may be used at any stage.
- The system must work with the official mock dataset without internet access to external data providers (a hosted language model is allowed).
- Plagiarism or reuse of another participant's code may lead to disqualification.

Appendix A. Mock Dataset Structure

The organisers distribute five JSON files. Field names are fixed so that all participants work with the same data. Currency is AZN. Dates use ISO 8601. Flight times are local to the airport.

File	Records	Main fields
flights.json	17	id, airline, flight_number, origin, destination, departure_local, arrival_local, duration_min, stops, via, price_azn, cabin, baggage_kg, seats_available
products.json	12	id, name (en, az, ru), category, brand, price_azn, rating, stock, color, description
movies.json	8	id, title, genre, language, duration_min, age_rating, rating, synopsis
cinemas.json	3	id, name, district, address
showtimes.json	16	id, movie_id, cinema_id, date, time, hall, price_azn, seats_total, seats_available

Example flight record:

```
{
  "id": "F001",
  "airline": "Caspian Air",
  "flight_number": "CA101",
  "origin": {"code": "GYD", "city": "Baku"},
  "destination": {"code": "IST", "city": "Istanbul"},
  "departure_local": "2026-11-06T07:30",
  "arrival_local": "2026-11-06T10:05",
  "duration_min": 215,
  "stops": 0,
  "via": null,
  "price_azn": 289.00,
  "cabin": "economy",
  "baggage_kg": 20,
  "seats_available": 14
}
```

Example product record:

```
{
  "id": "P001",
  "name": {"en": "Wireless Headphones", "az": "Simsiz qulaqliq", "ru": "Besprovodnye naushniki"},
  "category": "audio",
  "brand": "Sonivo",
  "price_azn": 89.90,
  "rating": 4.4,
  "stock": 23,
  "color": "black"
}
```

The Azerbaijani and Russian names in the distributed files use the full native alphabets. The example above is transliterated for this document only.